CPE 314 — Software Design

University of Cebu Computer Engineering Department

FleetRent: Vehicle Rental & Fleet Management System

Project Design Document

Group Name / No.:	Group 4
Members:	Gabasa, Dexter
	Rosal, Wendil
	Coloscos, Brent
	Lagaya, John Carlo
	Labra, Simone
Repository URL:	https://github.com/ColoscosBrent/cpe314-69323-capstone-team-04
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	0.1 / October 21, 2026

Table of Contents

Revision History 3
Formatting Guidelines 3
1. Introduction & Project Overview 3
2. Preliminary ER Sketch & Team Roles 4
3. System Architecture & Object-Oriented Design 5
4. Data Model & Database Design 5
5. API Integration Design 5
6. Concurrency Design 7
7. Data Mining & Visualization Design 7
8. Testing Summary 7
9. Individual Contributions 7
Appendix 8

Revision History

Table 1: Revision history of this document

Version	Date	Author	Changes
0.1	October 21, 2026	Team 4	Initial draft — Week 8 sections
0.2			Week 9 sections added
0.3			Week 10 sections added
1.0			Final compilation and polish — Week 11

Formatting Guidelines

Follow these formatting rules consistently throughout the document so the final compiled version reads as one polished deliverable, not four stitched-together drafts.

Table 2: Formatting guidelines for this document

Element	Rule
Body font	Calibri, 11pt (the default font of this template — do not mix fonts)
Headings	Use Word's built-in Heading 1/2/3 styles (already applied to every section title below) — this is required for the Table of Contents field to work
Margins	1 inch on all sides
Line spacing	1.15–1.5 line spacing for body paragraphs; single-spacing inside tables
Table captions	Label every table "Table N: short description" directly ABOVE the table, numbered sequentially from Table 1 through the end of the document
Figure/chart captions	Label every chart/screenshot "Figure N: short description" directly BELOW the figure, numbered sequentially from Figure 1
Code snippets	Use a monospace font (Consolas or Courier New), 9–10pt, and keep each snippet under ~20 lines — link to the full file in your repository instead of pasting it all
Page numbers	Already automatic — every page footer includes a live page number field
Alignment	Body text left-aligned/justified; headings left-aligned; do not center body paragraphs

1. Introduction & Project Overview

Our app, FleetRent, is a vehicle rental and fleet management desktop app built with JavaFX. It is for small rental shops that rent out cars, SUVs, and motorcycles. We chose Theme Q: Vehicle Rental & Fleet Management. Many small shops still write their rentals on paper, so it is easy to lose track of which vehicle is out, how many miles it has driven, and who is next in line. FleetRent keeps the vehicles, renters, and rentals in one SQLite database, keeps a waiting list when a vehicle is already out, and lets staff undo the last check-out. Real make and model data comes from the NHTSA vPIC API, and a dashboard shows the most rented vehicle type and how busy the fleet is over time.

2. Preliminary ER Sketch & Team Roles

This is the preliminary ER sketch from our Week 8 project proposal. It has three entities (Renter, Vehicle, and Rental) and two relationships. It is a rough draft and will be replaced by the finalized ERD in Section 4.

Table 3: Preliminary entities and their key attributes

Entity	Key attributes
Renter	id (PK), name, contact
Vehicle	id (PK), type (SEDAN | SUV | MOTORCYCLE), make, model, year, dailyRate, mileage, categoryPath
Rental	id (PK), renterId (FK to Renter.id), vehicleId (FK to Vehicle.id), startDate, endDate, milesDriven

Relationships: One Renter can have many Rentals (1:M), and one Vehicle can be in many Rentals over time (1:M). The Rental table connects a renter to a vehicle. The Queue (waiting list) and the Stack (undo) are kept in memory only, so they are not tables. categoryPath is a text such as "Car > Sedan > Compact Sedan" that we use to build the vehicle category tree in our code.

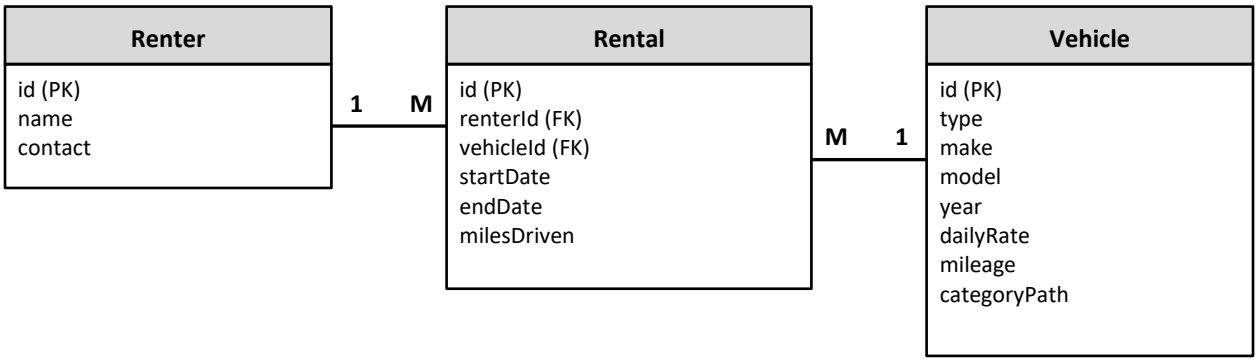


Figure 1: Preliminary ER sketch of FleetRent (Renter, Rental, Vehicle)

Team roster and primary roles: Each member has one main job for Milestone 1, the same as in our project proposal.

Table 4: Team roster and primary roles (Group 4)

Member (GitHub username)	Primary role / responsibility
Gabasa, Dexter (gabasad84-afk)	Database: SQLite tables for vehicles, renters, and rentals, and the JDBC code with PreparedStatement. Database tests.
Rosal, Wendil (wendilrosalx22)	Vehicle category tree and the recursive fleet mileage method. Maven project setup. Tests for these.
Coloscos, Brent (ColoscosBrent)	Domain model and OOD: Vehicle abstract class, its 3 subclasses, and the rate and fee logic. Tests for these.
Lagaya, John Carlo (caloylagaya2k4-pixel)	Rental and Renter classes, the waiting list Queue, the undo Stack, and the CSV import on a background thread. Tests for these.
Labra, Simone (simonelabra21)	NHTSA vPIC API: sending the request and parsing the JSON. JavaFX GUI (forms, buttons, dashboard) and the bar and line charts.
All members	Commit your own work to the repo, review each other's pull requests, and attend the Week 8-9 consultation.

3. System Architecture & Object-Oriented Design

Present your class hierarchy (a diagram or a clear textual description) showing the abstract class/interface and its concrete subclasses, and justify how each class follows the Single Responsibility Principle. Explain the design rationale in a short paragraph.

4. Data Model & Database Design

Include the finalized ER diagram (3+ entities, at least one relationship with correct cardinality) and the actual SQLite schema (CREATE TABLE statements or an equivalent schema table). This section supersedes the preliminary sketch from Section 2.

5. API Integration Design

Name the public API your app consumes, describe the endpoint(s) called, the shape of the JSON response, and how that data is parsed and used meaningfully in the app.

6. Concurrency Design

Identify the long-running operation(s) that run on a background thread, explain the threading approach used, and describe how the UI stays responsive during that operation.

7. Data Mining & Visualization Design

Describe the aggregation or frequency-analysis step performed on your app's data and the chart(s) used to visualize the result, including why that chart type was chosen.

8. Testing Summary

Summarize your JUnit test suite — how many tests exist, what they cover, and at least one meaningful edge case per tested class. A short table or list of key test cases is helpful here.

9. Individual Contributions

List what each team member specifically built or owned, referencing their git commit history, so the instructor can cross-check contribution during the Individual Q&A.

Appendix

(Add any additional references, links to full API documentation, licensing notes, or supplementary material here.)